

Cisco Packet Tracer Lab Practicals — CCNA-Level Study Guide

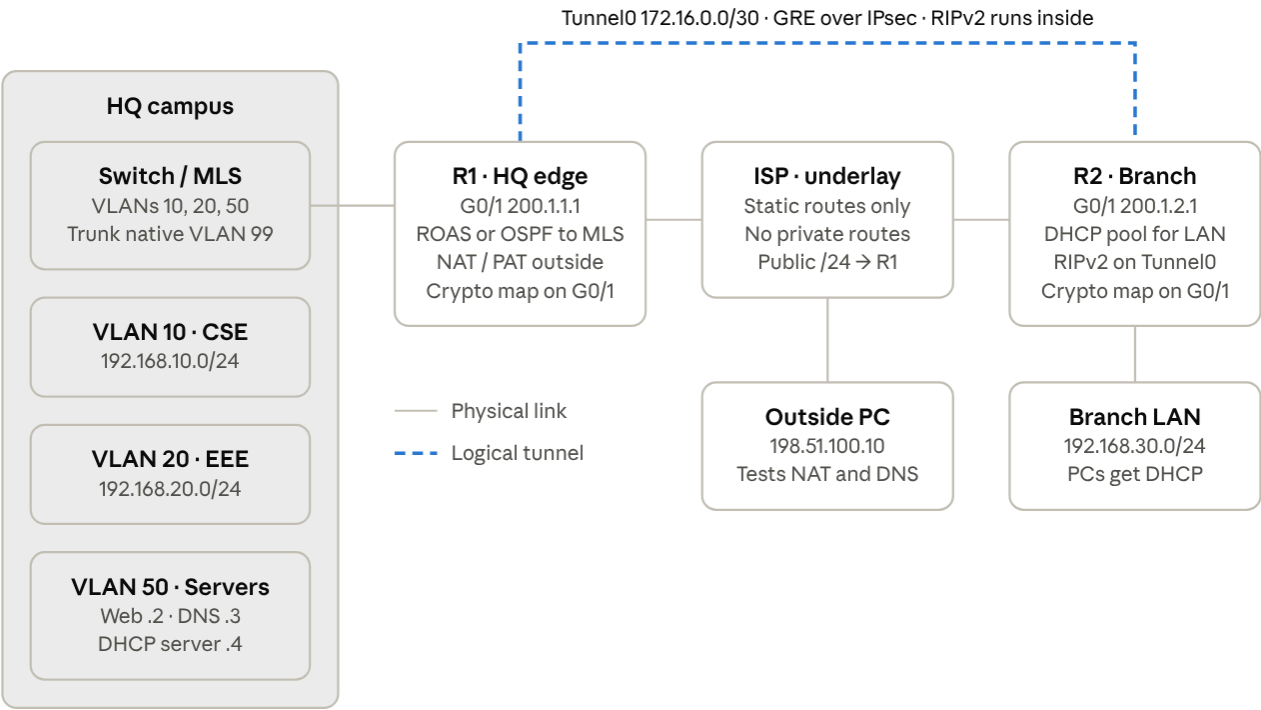
Sep 28, 2026 · @Mahi

Reference lab topology

Every example in this guide uses one addressing plan: an HQ campus behind edge router R1, a Branch behind R2, and an ISP that carries only public routes between them.

HQ and Branch reach each other through a tunnel riding the ISP

Reference lab used by every example in this guide



Reference lab · HQ campus, R1 edge, ISP underlay, Branch, GRE/IPsec overlay

Read it left to right: hosts live in VLANs, R1 routes them out through NAT, and the dashed tunnel is the private overlay that joins the HQ and Branch LANs across the ISP.

Segment	Network	Key addresses
VLAN 10 — CSE PCs	192.168.10.0/24	Gateway .1, PCs from .10
VLAN 20 — EEE PCs	192.168.20.0/24	Gateway .1, PCs from .10
VLAN 50 — Servers	192.168.50.0/24	Gateway .1, Web .2, DNS .3, DHCP server .4

Segment	Network	Key addresses
VLAN 99 — Native (unused)	none	No hosts; trunk native VLAN only
VLAN 999 — Blackhole	none	Parking VLAN for unused, shut ports
R1 ↔ MLS (Design B only)	10.0.0.0/30	R1 .1, MLS .2
R1 ↔ ISP	200.1.1.0/30	R1 .1, ISP .2
R2 ↔ ISP	200.1.2.0/30	R2 .1, ISP .2
Branch LAN	192.168.30.0/24	R2 .1, PCs via DHCP from .10
Tunnel0 (overlay)	172.16.0.0/30	R1 .1, R2 .2
KUET public block (NAT)	203.0.113.0/24, routed by the ISP to R1	Servers published on 203.0.113.5; PAT pool .20–.30
Outside test network	198.51.100.0/24	ISP .1, Outside PC .10

Two HQ designs appear. **Design A** does inter-VLAN routing on R1 subinterfaces (router-on-a-stick), so R1 owns the VLAN gateways directly; the NAT, GRE, RIPv2 and VPN examples assume it. **Design B** moves the gateways onto a multilayer switch (SVIs) and joins it to R1 with a routed /30 running OSPF; the OSPF and DHCP-relay examples assume it. Part 1 compares the two.

Part 1 — Layer-2 and Layer-3 switching

A VLAN is a separate broadcast domain on shared switch hardware; hosts in different VLANs can only talk through a Layer-3 device, which is why every VLAN design ends with inter-VLAN routing.

1.1 VLANs: what they are and why you create them

A switch floods broadcasts (ARP requests, DHCP Discovers) out every port in the same VLAN. Putting CSE in VLAN 10 and EEE in VLAN 20 means a CSE broadcast never reaches EEE: less noise, faults stay contained, and you can apply security per group. Map each VLAN to exactly one IP subnet — that one-to-one rule is what makes routing between them possible.

```

! ===== SW1: create and name VLANs (global configuration) =====
SW1(config)# vlan 10
SW1(config-vlan)# name CSE
SW1(config-vlan)# vlan 20
SW1(config-vlan)# name EEE
SW1(config-vlan)# vlan 50
SW1(config-vlan)# name SERVERS
SW1(config-vlan)# vlan 99
SW1(config-vlan)# name NATIVE_UNUSED      ! trunk native VLAN only - never
give it hosts
SW1(config-vlan)# vlan 999
SW1(config-vlan)# name BLACKHOLE          ! parking VLAN for unused ports
SW1(config-vlan)# exit

```

VLAN 1 is the default: every port starts in it and it cannot be deleted. Best practice is to move every host and every unused port off VLAN 1.

1.2 Access ports

An access port belongs to exactly one VLAN and sends and receives untagged frames, so the PC never knows VLANs exist. Use access ports for every end device: PCs, printers, servers, access points in simple labs.

```

SW1(config)# interface range fa0/1 - 10
SW1(config-if-range)# switchport mode access      ! hard-code access: no DTP
trunk negotiation
SW1(config-if-range)# switchport access vlan 10    ! port joins VLAN 10 (VLAN
is auto-created if missing)
SW1(config-if-range)# spanning-tree portfast      ! host port: skip ~30 s of
STP listening/learning
SW1(config-if-range)# exit
SW1(config)# interface range fa0/11 - 20
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 20
SW1(config-if-range)# exit
SW1(config)# interface fa0/21
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 50         ! server port
SW1(config-if)# spanning-tree portfast

```

PortFast belongs only on ports facing end devices. On a port facing another switch it can create a temporary loop.

1.3 Trunk ports and 802.1Q

A trunk carries many VLANs over one link. The sending switch inserts a 4-byte 802.1Q tag holding the 12-bit VLAN ID (1–4094) into each frame; the receiver reads the tag, strips it and forwards the frame only inside that VLAN. Use trunks on switch-to-switch links, on the switch-to-router link for router-on-a-stick, and toward virtualised servers.

```
SW1(config)# interface gi0/1
SW1(config-if)# switchport trunk encapsulation dot1q      ! 3560/3650 MLS
only; a 2960 supports only dot1q and rejects this line
SW1(config-if)# switchport mode trunk                    ! static trunk
SW1(config-if)# switchport trunk native vlan 99          ! untagged frames
belong to VLAN 99, not VLAN 1
SW1(config-if)# switchport trunk allowed vlan 10,20,50,99 ! prune: only
needed VLANs cross this link
SW1(config-if)# switchport nonegotiate                  ! stop sending DTP
frames
SW1(config-if)# exit
```

Why the encapsulation line matters on a multilayer switch: it supports both ISL and dot1q, so the port sits in auto encapsulation and IOS rejects `switchport mode trunk` until you choose one.

To add a VLAN later, use `switchport trunk allowed vlan add 30`. Retyping the list without `add` replaces it — a classic way to break a working trunk.

1.4 Native VLAN security

The native VLAN is the one VLAN whose frames cross a trunk untagged. By default it is VLAN 1 — the same VLAN every unconfigured port sits in. Two attacks exploit this.

- **VLAN hopping by double tagging.** An attacker in the native VLAN sends a frame with two tags: outer = native VLAN, inner = victim VLAN. The first switch strips the outer tag (it is native, so it goes out untagged) and forwards the frame down the trunk. The next switch reads the inner tag and delivers it into the victim VLAN. It is one-way, but it works.
- **Switch spoofing.** A port left in dynamic mode negotiates a trunk (DTP) with the attacker's device, which then sees every VLAN.

Hardening step	Command	What it stops
Native VLAN = unused VLAN with no hosts	<code>switchport trunk native vlan 99</code>	Double tagging: no attacker can sit in VLAN 99

Hardening step	Command	What it stops
Hard-code every port's mode	<code>switchport mode access</code> or <code>switchport mode trunk</code>	Switch spoofing
Disable DTP on trunks	<code>switchport nonegotiate</code>	DTP negotiation
Prune each trunk	<code>switchport trunk allowed vlan ...</code>	Unneeded VLANs reaching a link
Park and shut unused ports	<code>switchport access vlan 999</code> + <code>shutdown</code>	Rogue devices plugged into spare ports
Keep hosts off VLAN 1	Move every port to a named VLAN	Default-VLAN attacks

```
SW1(config)# interface range fa0/22 - 24, gi0/2
SW1(config-if-range)# switchport mode access
SW1(config-if-range)# switchport access vlan 999    ! blackhole VLAN
SW1(config-if-range)# shutdown
```

The native VLAN must match on both ends of a trunk. A mismatch still passes tagged traffic, but untagged frames leak between the two different native VLANs and CDP logs `%CDP-4-NATIVE_VLAN_MISMATCH` repeatedly.

Verification

```
SW1# show vlan brief                ! access ports per VLAN (trunk ports
are NOT listed here)
SW1# show interfaces trunk          ! mode, encapsulation, status, native
VLAN, allowed and active VLANs
SW1# show interfaces fa0/1 switchport ! administrative vs operational mode,
access VLAN, native VLAN
SW1# show mac address-table         ! which MAC was learned on which port
and VLAN
```

In `show interfaces trunk`, the last block (VLANs in spanning tree forwarding state and not pruned) is what really crosses the link. A VLAN missing there either does not exist on this switch, is not allowed, or is blocked by STP.

1.5 Inter-VLAN routing: why a Layer-3 device is needed

A PC in VLAN 10 sending to 192.168.20.10 sees a different subnet, so it addresses the frame to its default gateway's MAC. Some Layer-3 device must own an IP in every VLAN and route between them. The labs use two designs.

Design A — Router-on-a-Stick (ROAS)

One physical router interface is split into logical subinterfaces, one per VLAN. The switch port facing the router is a trunk; each subinterface is told which 802.1Q tag it owns and becomes that VLAN's gateway.

```
! ===== SW1: the link to the router MUST be a trunk =====
SW1(config)# interface gi0/1
SW1(config-if)# switchport mode trunk
SW1(config-if)# switchport trunk native vlan 99

! ===== R1 =====
R1(config)# interface gi0/0
R1(config-if)# no shutdown                                ! parent up; subinterfaces
follow its state
R1(config-if)# exit
R1(config)# interface gi0/0.10                            ! .10 is only a label;
matching the VLAN keeps it readable
R1(config-subif)# description Gateway for CSE
R1(config-subif)# encapsulation dot1Q 10                  ! MUST come before the IP
address
R1(config-subif)# ip address 192.168.10.1 255.255.255.0
R1(config-subif)# interface gi0/0.20
R1(config-subif)# encapsulation dot1Q 20
R1(config-subif)# ip address 192.168.20.1 255.255.255.0
R1(config-subif)# interface gi0/0.50
R1(config-subif)# encapsulation dot1Q 50
R1(config-subif)# ip address 192.168.50.1 255.255.255.0
R1(config-subif)# interface gi0/0.99
R1(config-subif)# encapsulation dot1Q 99 native           ! match the switch's native
VLAN; no IP needed
R1(config-subif)# end
```

IOS rejects an IP address on a subinterface that has no encapsulation yet, so always type the `encapsulation dot1Q` line first.

Packet path from PC-A (VLAN 10) to PC-B (VLAN 20): the switch tags the frame 10 and sends it up the trunk; R1 receives it on G0/0.10, looks up 192.168.20.0/24, and sends it out G0/0.20 tagged 20; the frame comes back down the same trunk and the switch delivers it untagged to PC-B. Every inter-VLAN packet crosses the single link twice — that is the design's bottleneck.

Design B — Multilayer switch routing (SVIs)

A multilayer switch (3560/3650 in Packet Tracer) contains a routing engine. A Switch Virtual Interface (`interface vlan X`) is a logical Layer-3 port for a whole VLAN. With `ip routing` enabled, the switch routes between SVIs in hardware (ASICs) at wire speed.

```

MLS1(config)# ip routing                                ! OFF by default: without
it SVIs are only management IPs
MLS1(config)# vlan 10
MLS1(config-vlan)# vlan 20
MLS1(config-vlan)# vlan 50
MLS1(config-vlan)# exit
MLS1(config)# interface vlan 10
MLS1(config-if)# ip address 192.168.10.1 255.255.255.0    ! CSE gateway
MLS1(config-if)# no shutdown
MLS1(config-if)# interface vlan 20
MLS1(config-if)# ip address 192.168.20.1 255.255.255.0    ! EEE gateway
MLS1(config-if)# no shutdown
MLS1(config-if)# interface vlan 50
MLS1(config-if)# ip address 192.168.50.1 255.255.255.0    ! server gateway
MLS1(config-if)# no shutdown
MLS1(config-if)# exit
! ----- routed port toward R1: behaves exactly like a router interface -----
MLS1(config)# interface gi0/1
MLS1(config-if)# no switchport                          ! convert the Layer-2 port
into a Layer-3 routed port
MLS1(config-if)# ip address 10.0.0.2 255.255.255.252
MLS1(config-if)# no shutdown
MLS1(config-if)# exit
MLS1(config)# ip route 0.0.0.0 0.0.0.0 10.0.0.1          ! temporary; OSPF replaces
this in Part 3

```

An SVI is up/up only when three things are true: the VLAN exists in the VLAN database, the SVI is `no shutdown`, and at least one port in that VLAN (an access port, or a trunk allowing it) is up and forwarding. An SVI stuck at up/down almost always means the VLAN has no active port.

Technical comparison

Factor	Router-on-a-Stick	Multilayer switch (SVI)
Where routing happens	Router CPU, subinterfaces	Switch hardware (ASIC), SVIs
Gateway configured as	<code>interface g0/0.X + encapsulation dot1Q X</code>	<code>interface vlan X + ip routing</code>

Factor	Router-on-a-Stick	Multilayer switch (SVI)
Link to the access layer	One trunk carries every VLAN, both directions	VLANs are local to the switch
Throughput	Limited by one link and the router CPU	Wire speed across the switch backplane
Single point of failure	The trunk and the router	The switch (fixed with a redundant pair)
Hardware needed	Any router plus a basic L2 switch	A Layer-3 switch
Other features on the same box	NAT, VPN, WAN interfaces	Fast LAN routing; usually no NAT or VPN
Best for	Small sites, branches, labs, few VLANs	Campus core/distribution, many VLANs, heavy traffic between VLANs

Where to use what. Choose ROAS when you have one router, a handful of VLANs and modest traffic — a branch like R2. Choose an MLS when many VLANs exchange a lot of traffic, such as a university campus, and keep the router at the edge for NAT and VPN, joined by a routed link. That is exactly Design B: MLS for the campus, R1 for the Internet edge.

Verification

```

R1# show ip interface brief      ! every subinterface or SVI: IP, Status up,
Protocol up
R1# show vlans                  ! (router) which 802.1Q VLAN each
subinterface owns, with counters
R1# show ip route connected      ! one C and one L route per VLAN subnet
MLS1# show ip route              ! must list C routes; an empty table means ip
routing is missing
MLS1# show interfaces vlan 10    ! SVI state
PC> ping 192.168.20.10          ! the first reply may time out while ARP
resolves: normal

```

Part 2 — IP services, addressing and Packet Tracer specifics

VLSM gives each subnet only the block size it needs, DHCP hands out the addresses inside those blocks automatically, and DNS lets users reach servers by name instead of by IP.

2.1 VLSM subnetting

Core rule: allocate from the largest requirement to the smallest. Block sizes are powers of two, and every subnet must start on a multiple of its own block size. Placing big blocks first keeps every later, smaller block aligned automatically. Placing small blocks first leaves gaps a big block cannot fit into, which wastes space or causes overlap.

1. Sort the subnets by required hosts, largest first. Point-to-point WAN links (2 hosts) go last.
2. For each, find the smallest n where $2^n - 2 \geq \text{hosts}$. The -2 removes the network and broadcast addresses.
3. Block size = 2^n ; prefix = $/(32 - n)$.
4. Start at the next free address. Going largest to smallest, it is always a legal boundary.
5. Network = start; broadcast = start + block - 1; usable = everything between.

Hosts needed	Block	Prefix	Subnet mask	Wildcard
up to 2	4	/30	255.255.255.252	0.0.0.3
up to 6	8	/29	255.255.255.248	0.0.0.7
up to 14	16	/28	255.255.255.240	0.0.0.15
up to 30	32	/27	255.255.255.224	0.0.0.31
up to 62	64	/26	255.255.255.192	0.0.0.63
up to 126	128	/25	255.255.255.128	0.0.0.127
up to 254	256	/24	255.255.255.0	0.0.0.255

Worked example — 192.168.1.0/24 for four KUET departments: CSE 100 hosts, EEE 50, ME 25, Admin 10, plus two WAN links.

Subnet	Hosts	Block	Network / prefix	Usable range	Broadcast
CSE	100	128	192.168.1.0/25	.1 - .126	.127
EEE	50	64	192.168.1.128/26	.129 - .190	.191
ME	25	32	192.168.1.192/27	.193 - .222	.223
Admin	10	16	192.168.1.224/28	.225 - .238	.239
WAN 1	2	4	192.168.1.240/30	.241 - .242	.243
WAN 2	2	4	192.168.1.244/30	.245 - .246	.247
Free	—	—	192.168.1.248 - .255	Spare for growth	—

Sanity check: each network address must divide evenly by its block — $128 \div 64 = 2$, $192 \div 32 = 6$, $224 \div 16 = 14$, $240 \div 4 = 60$. A remainder means the subnet sits on an illegal boundary.

Address assignment logic. The nth usable address = network address + n. Labs fix each role to a position so anyone can predict an address without looking it up.

Role	Position	CSE /25	EEE /26	ME /27	Admin /28
Default gateway	1st usable	.1	.129	.193	.225
Server	2nd usable	.2	.130	.194	.226
First PC	10th usable	.10	.138	.202	.234
Last usable	broadcast - 1	.126	.190	.222	.238

The common slip: EEE's 10th usable is $128 + 10 = .138$, not .10 and not .139. The positions also drive DHCP — exclude the 1st to 9th usable addresses (gateway, servers, printers) so the pool starts at the 10th.

2.2 How DHCP works (DORA)

1. **Discover** — the client broadcasts: source 0.0.0.0, destination 255.255.255.255, UDP 68 to 67.
2. **Offer** — a server proposes an address, mask, gateway, DNS server and lease time.
3. **Request** — the client broadcasts that it accepts one offer, so any other server withdraws its own.
4. **Acknowledge** — the server confirms and the client configures itself.

Discover and Request are broadcasts, and routers never forward broadcasts. So a DHCP server only serves its own subnet unless a relay agent helps — every configuration below follows from that one fact.

2.3 DHCP on an IOS router (Branch R2)

Use this when the router is already the gateway of a small site and there is no separate server.

```

R2(config)# ip dhcp excluded-address 192.168.30.1 192.168.30.9 ! 1st-9th
usable: gateway, servers, printers
R2(config)# ip dhcp pool BRANCH_LAN
R2(dhcp-config)# network 192.168.30.0 255.255.255.0 ! the subnet this pool
serves
R2(dhcp-config)# default-router 192.168.30.1 ! client gateway = R2
G0/0
R2(dhcp-config)# dns-server 192.168.50.3 ! HQ DNS server,
reached through the tunnel
R2(dhcp-config)# domain-name kuet.ac.bd ! optional
R2(dhcp-config)# exit
R2(config)# interface gi0/0
R2(config-if)# ip address 192.168.30.1 255.255.255.0
R2(config-if)# no shutdown

```

- `ip dhcp excluded-address` is a global command, not a pool command. Configure it before the pool: the router starts leasing the moment a pool exists, and a PC could grab the gateway's address.
- The pool's `network` must match the subnet of the interface that receives the request (or the relay's giaddr, below). If nothing matches, no offer is sent.
- `default-router` is the single most important option: without it clients get an address but cannot leave their subnet.

```

R2# show ip dhcp binding ! leases handed out: IP and client MAC
R2# show ip dhcp pool ! pool size, leased count, next address to
offer
R2# show ip dhcp conflict ! addresses found already in use
R2# clear ip dhcp binding * ! wipe leases while testing

```

2.4 Cross-topology DHCP relay (ip helper-address)

In Design B the DHCP server (192.168.50.4) sits in VLAN 50, but the clients are in VLANs 10 and 20. Their broadcasts stop at their gateway. A relay agent on that gateway catches the broadcast and forwards it to the server as unicast.

1. The gateway receives the client's broadcast on the client-facing interface.
2. It writes that interface's IP into the packet's **giaddr** (gateway IP address) field — for example 192.168.10.1.
3. It unicasts the packet to the helper address.
4. The server picks the pool whose subnet contains the giaddr and replies to the relay, which passes the offer to the client.

So the server needs one pool per client subnet, each with that subnet's own gateway, plus a route back to every giaddr.

```
! ===== Design B: on the MLS SVIs (the clients' gateways) =====
MLS1(config)# interface vlan 10
MLS1(config-if)# ip helper-address 192.168.50.4      ! the DHCP server's IP
MLS1(config-if)# interface vlan 20
MLS1(config-if)# ip helper-address 192.168.50.4
! No helper on VLAN 50: the server is local there and hears broadcasts
directly

! ===== Design A: the same idea on R1's subinterfaces =====
R1(config)# interface gi0/0.10
R1(config-subif)# ip helper-address 192.168.50.4
R1(config-subif)# interface gi0/0.20
R1(config-subif)# ip helper-address 192.168.50.4

! ===== Remote relay: Branch clients, HQ server (works once the overlay
routes 192.168.50.0 to R2) =====
R2(config)# interface gi0/0
R2(config-if)# ip helper-address 192.168.50.4
R2(config-if)# exit
R2(config)# no ip dhcp pool BRANCH_LAN                ! remove the local pool:
one server per subnet
```

Rule of thumb: `ip helper-address` goes on the interface where the clients' broadcast **arrives** — their gateway — never on the server-side interface. Besides DHCP, it also forwards a few other UDP broadcasts by default (TFTP, DNS, Time, NetBIOS, TACACS), which is harmless in labs.

2.5 Dedicated DHCP server device in Packet Tracer

1. Place a **Server-PT**. Desktop → IP Configuration → Static: IP 192.168.50.4, mask 255.255.255.0, gateway 192.168.50.1, DNS 192.168.50.3. The gateway is mandatory: replies to relayed requests go to other subnets.
2. Services tab → DHCP → set Service to **On**.
3. Edit the default **serverPool** so it matches VLAN 50 and click **Save**.
4. For each remote subnet, type a new Pool Name, fill the fields and click **Add**.

Field	VLAN 10 pool	VLAN 20 pool	Branch pool
Pool Name	CSE_POOL	EEE_POOL	BRANCH_POOL
Default Gateway	192.168.10.1	192.168.20.1	192.168.30.1

Field	VLAN 10 pool	VLAN 20 pool	Branch pool
DNS Server	192.168.50.3	192.168.50.3	192.168.50.3
Start IP Address	192.168.10.10	192.168.20.10	192.168.30.10
Subnet Mask	255.255.255.0	255.255.255.0	255.255.255.0
Maximum Number of Users	245	245	245

The GUI has no exclusion list. You exclude by starting the pool at the 10th usable and capping Maximum Users so it stops before the broadcast ($.10 + 245 - 1 = .254$).

5. On each PC: Desktop → IP Configuration → **DHCP**. Success shows an address, gateway and DNS. Failure shows a DHCP request failed message and a 169.254.x.x (APIPA) address.

Situation	Best choice
Small branch, router is already the gateway	IOS router pool
Many VLANs, central management	Dedicated server + <code>ip helper-address</code> on every client gateway
Server in another subnet or site	Relay is mandatory
Server in the same VLAN as the clients	No relay needed

2.6 DNS: A-records that point to the public NAT address

An A-record maps a name to an IPv4 address; the design question is which address. The web server is 192.168.50.2 inside, but outside users reach it through static NAT as 203.0.113.5 (Part 4). A private 192.168.x.x address is not routable on the Internet, so an outside client handed 192.168.50.2 would send its packets nowhere. The record must return the **inside global** (public) address.

On the DNS server (192.168.50.3): Services → DNS → **On**, then add each record with **Add**.

Name	Type	Address
www.kuet.ac.bd	A Record	203.0.113.5
dns.kuet.ac.bd	A Record	203.0.113.5

End-to-end flow for the Outside PC (198.51.100.10), whose DNS server is set to 203.0.113.5:

1. The PC sends a DNS query (UDP 53) to 203.0.113.5.
2. R1's static NAT turns 203.0.113.5:53 into 192.168.50.3:53; the server answers that `www.kuet.ac.bd` is 203.0.113.5.
3. The browser opens `http://www.kuet.ac.bd` — a TCP 80 connection to 203.0.113.5.
4. R1 turns 203.0.113.5:80 into 192.168.50.2:80, and the page loads.

Caveat for inside clients: an HQ PC that resolves `www` to 203.0.113.5 would have to leave through R1 and come straight back in (NAT hairpinning), which Packet Tracer does not handle reliably. Inside clients should browse by the private IP or use a separate internal record — real networks solve this with split DNS.

2.7 CLI DNS lookups, typos and Ctrl+Shift+6

In user or privileged EXEC mode, IOS treats any word it does not recognise as a hostname you want to Telnet to. Type `shwo ip route` and the router tries to resolve `shwo` by broadcasting a DNS query to 255.255.255.255, prints a Translating ... domain server (255.255.255.255) message and freezes until the lookup times out.

```
Router# ! stuck in a lookup, ping or traceroute? press Ctrl+Shift+6
(sometimes twice) to abort
Router# configure terminal
Router(config)# no ip domain-lookup                ! typos now fail instantly
instead of freezing
Router(config)# line console 0
Router(config-line)# logging synchronous           ! log messages no longer break
the line you are typing
Router(config-line)# exec-timeout 10 0              ! log out after 10 idle minutes
(0 0 = never; labs only)
```

- **Trade-off:** with `no ip domain-lookup` the router can no longer resolve names itself, so `ping www.kuet.ac.bd` from the router fails. If you need names on the router, keep lookups on and point it at a real server: `ip domain-lookup` plus `ip name-server 192.168.50.3`.
- Client PCs are unaffected either way; the command changes only the router's own CLI.
- Do not confuse it with `ip domain-name kuet.ac.bd`, which SSH needs to build its RSA key (Part 5). The two are independent: you can set a domain name and still disable lookups.

Part 3 — Routing: static, OSPF, RIPv2, underlay vs overlay

A router forwards only to networks in its routing table; static routes, OSPF and RIPv2 are three ways of filling that table, and the labs use each where it fits best.

3.1 How the router chooses a route

The longest matching prefix always wins first: a /24 beats a /16 beats the /0 default. When two sources offer the same prefix, the lower administrative distance (AD) — how much the router trusts the source — wins.

Source	Code	AD
Directly connected	C	0
Static	S	1
OSPF	O	110
RIP	R	120

3.2 Static routing

A static route is a manual instruction: to reach network X, send to next hop Y. It costs no CPU or bandwidth, but it never adapts on its own. Use it for stub sites with one way out, for the default route toward the ISP, and for the underlay that tunnels depend on.

```
! Default route: anything not in the table goes to the ISP
R1(config)# ip route 0.0.0.0 0.0.0.0 200.1.1.2      ! next-hop form: preferred
on Ethernet
R1(config)# ip route 0.0.0.0 0.0.0.0 gi0/1          ! exit-interface form:
fine on serial, avoid on Ethernet
! Floating static: append an AD higher than the dynamic protocol, e.g. ...
<backup next hop> 130
! It stays hidden until the OSPF or RIP route disappears

! The ISP knows only public space: its own links plus the block KUET owns
ISP(config)# ip route 203.0.113.0 255.255.255.0 200.1.1.1 ! KUET public
block -> R1 (NAT return traffic)
```

On Ethernet, the exit-interface form makes the router ARP for every single destination, so use a next-hop address there.

3.3 OSPF (link-state)

OSPF routers exchange link-state advertisements (LSAs) until every router in an area holds the same map, then each runs Dijkstra's SPF algorithm to pick the lowest-cost path. Cost = reference bandwidth (100 Mbps by default) ÷ interface bandwidth. OSPF converges fast, carries subnet masks (so VLSM works) and scales through areas. Use it inside the enterprise — here between the MLS and R1 in Design B.

```

! ===== R1 (HQ edge) =====
R1(config)# ip route 0.0.0.0 0.0.0.0 200.1.1.2      ! the default route OSPF
will advertise: it must exist
R1(config)# router ospf 10                          ! process ID 10: local
only, 1-65535
R1(config-router)# router-id 1.1.1.1                ! stable, readable ID;
set before neighbours form
R1(config-router)# network 10.0.0.0 0.0.0.3 area 0  ! run OSPF on interfaces
inside 10.0.0.0/30
R1(config-router)# default-information originate     ! inject R1's default
route into OSPF
R1(config-router)# exit

! ===== MLS1 =====
MLS1(config)# no ip route 0.0.0.0 0.0.0.0 10.0.0.1 ! remove the temporary
static from Part 1
MLS1(config)# router ospf 10
MLS1(config-router)# router-id 2.2.2.2
MLS1(config-router)# network 10.0.0.0 0.0.0.3 area 0
MLS1(config-router)# network 192.168.10.0 0.0.0.255 area 0
MLS1(config-router)# network 192.168.20.0 0.0.0.255 area 0
MLS1(config-router)# network 192.168.50.0 0.0.0.255 area 0
MLS1(config-router)# passive-interface vlan 10      ! advertise the subnet,
send no hellos to PCs
MLS1(config-router)# passive-interface vlan 20
MLS1(config-router)# passive-interface vlan 50

```

- **Process ID** (`router ospf 10`) names the OSPF instance on this router only. Neighbours with process IDs 1 and 10 still become adjacent.
- **network + wildcard + area** does not advertise a network directly. It means: every interface whose IP matches this address and wildcard joins OSPF in this area; OSPF then advertises that interface's real subnet. Wildcard = 255.255.255.255 – subnet mask; a 0 bit must match, a 1 bit is ignored. `network 10.0.0.2 0.0.0.0 area 0` matches exactly one interface — the most precise form.
- **Area** must match on both ends of a link. Single-area labs use area 0, the backbone; in multi-area designs every other area must connect to area 0.
- **Router ID** is chosen in this order: the `router-id` command, then the highest loopback IP, then the highest active interface IP. Changing it later takes effect only after `clear ip ospf process`.
- **passive-interface** keeps a subnet advertised but stops hellos on LAN ports where no router lives. It saves overhead and stops a rogue router from peering.

default-information originate. R1 is the only router with a path to the Internet. Instead of typing a static default on every internal router, R1 advertises its own default route into OSPF as an external (type 5) LSA, and every other OSPF router installs it as `0*E2 0.0.0.0/0`. Two conditions: R1 must have a default route in its own table (unless you add the `always` keyword), and it must have an OSPF neighbour to tell. E2 means the metric stays at the seed value of 1 everywhere instead of adding internal cost.

Adjacency checklist — if `show ip ospf neighbor` is empty, compare both ends: same subnet and mask, same area, same hello/dead timers (10/40 s on Ethernet), same authentication, unique router IDs, interface not passive, same MTU.

```
R1# show ip ospf neighbor          ! FULL = adjacency complete
R1# show ip protocols              ! process ID, router ID, networks, passive
interfaces
R1# show ip route ospf             ! only OSPF routes
R1# show ip ospf interface brief   ! interfaces running OSPF, area, cost,
state
MLS1# show ip route                ! expect: 0*E2 0.0.0.0/0 [110/1] via
10.0.0.1
```

3.4 RIPv2 (distance-vector)

RIP routers send their whole table to their neighbours every 30 seconds; the metric is hop count, with 15 the maximum and 16 meaning unreachable. Version 2 adds what version 1 lacked: subnet masks in updates (so VLSM works), multicast updates to 224.0.0.9 instead of broadcasts, and authentication. Use it for small, simple networks and — as in this lab — as a lightweight protocol inside a GRE tunnel.

```
R2(config)# router rip
R2(config-router)# version 2          ! default sends v1 and accepts
both
R2(config-router)# no auto-summary     ! advertise real masks, not
classful summaries
R2(config-router)# network 172.16.0.0  ! classful: enables RIP on
Tunnel0 (172.16.0.2)
R2(config-router)# network 192.168.30.0 ! enables RIP on G0/0 and
advertises the branch LAN
R2(config-router)# passive-interface gi0/0 ! no updates toward the PCs
```

The RIP `network` command is classful: you type the major network (the class A, B or C boundary) with no mask or wildcard, and IOS enables RIP on every interface inside it. `network 172.16.0.0` covers every 172.16.x.x interface.

Why disable auto-summary. With it on, a router crossing a major-network boundary advertises only the classful summary. Suppose HQ and Branch both use subnets of 10.0.0.0 and are separated by 172.16.x.x links — a discontinuous network. Both sides advertise just 10.0.0.0/8, so each router sees two equal paths to 10.0.0.0/8 and half the traffic goes to the wrong site. `no auto-summary` makes RIPv2 send the real /24, /25 and /30 prefixes.

```
R2# show ip protocols      ! version sent/received, networks, auto-summary
                             state, neighbours
R2# show ip route rip      ! R routes shown as [120/hops]
R2# debug ip rip           ! watch updates in real time; stop with undebug all
```

3.5 Underlay vs overlay: static routes, a GRE tunnel and RIPv2

This is the central idea of the WAN lab (Design A: R1 owns 192.168.10.0, .20.0 and .50.0 directly on its subinterfaces).

The problem. HQ and Branch use private addresses. The ISP will never carry routes for private networks and should never learn your internal routes, so the two LANs cannot reach each other with normal routing across the ISP.

Underlay is the real, physical transport. Its only job is to let R1's WAN IP (200.1.1.1) reach R2's WAN IP (200.1.2.1). Static default routes toward the ISP do this, and the ISP only ever sees public 200.1.x.x addresses.

Overlay is a virtual network built on top. A GRE tunnel creates a logical point-to-point link, Tunnel0 (172.16.0.0/30), between R1 and R2. GRE wraps each original packet inside a new IP header — source 200.1.1.1, destination 200.1.2.1, IP protocol 47. The ISP routes on that outer header while the private LAN addresses ride inside, unseen. RIPv2 runs across Tunnel0 as if it were a cable between R1 and R2, so each router learns the other's LANs with the tunnel as next hop.

	Underlay	Overlay
What it is	Physical path R1 ↔ ISP ↔ R2	Logical Tunnel0 R1 ↔ R2
Addresses	200.1.1.0/30, 200.1.2.0/30	172.16.0.0/30 + private LANs
Routing	Static defaults to the ISP	RIPv2 over Tunnel0
Carries	GRE packets (outer header)	Original LAN packets (inner header)
Visible to	The ISP	Only R1 and R2

```

! ===== 1. Underlay: each router reaches the other's WAN IP =====
R1(config)# ip route 0.0.0.0 0.0.0.0 200.1.1.2
R2(config)# ip route 0.0.0.0 0.0.0.0 200.1.2.2
! Test before going further: R1# ping 200.1.2.1 (must succeed)

! ===== 2. Overlay link: GRE tunnel =====
R1(config)# interface tunnel 0
R1(config-if)# ip address 172.16.0.1 255.255.255.252
R1(config-if)# tunnel source gi0/1 ! the underlay interface
(or 200.1.1.1)
R1(config-if)# tunnel destination 200.1.2.1 ! R2's WAN IP, reached
through the underlay
R1(config-if)# tunnel mode gre ip ! the default; shown for
clarity
R1(config-if)# exit

R2(config)# interface tunnel 0
R2(config-if)# ip address 172.16.0.2 255.255.255.252
R2(config-if)# tunnel source gi0/1
R2(config-if)# tunnel destination 200.1.1.1
R2(config-if)# exit

! ===== 3. Overlay routing: RIPv2 on the tunnel and the LANs ONLY =====
R1(config)# router rip
R1(config-router)# version 2
R1(config-router)# no auto-summary
R1(config-router)# network 172.16.0.0 ! Tunnel0
R1(config-router)# network 192.168.10.0 ! CSE
R1(config-router)# network 192.168.20.0 ! EEE
R1(config-router)# network 192.168.50.0 ! Servers
! NO network 200.1.1.0 - the WAN must never be advertised into the overlay
! R2: network 172.16.0.0 and network 192.168.30.0, exactly as in 3.4

```

Never advertise the WAN networks into RIP. If R1 learned the route to 200.1.2.1 through RIP over the tunnel, the path to the tunnel's own destination would point into the tunnel — the tunnel would try to carry itself. IOS detects the loop, logs `%TUN-5-RECURDOWN: Tunnel0 temporarily disabled due to recursive routing`, and shuts the tunnel. The route to the tunnel destination must always come from the underlay (the statics).

- Tunnel0 shows up/up as soon as its source interface is up and a route to the destination exists — even if R2 has no tunnel yet. Prove it with `ping 172.16.0.2`.
- GRE adds 24 bytes (a 20-byte IP header plus 4 bytes of GRE). On real routers you would add `ip mtu 1400` and `ip tcp adjust-mss 1360` on the tunnel to avoid fragmentation.

- GRE carries multicast, so routing protocols run over it. GRE has no encryption: anyone on the ISP path can read the inner packets. Part 5 fixes that with IPsec.

```
R1# show interfaces tunnel 0      ! up/up, tunnel source and destination,
GRE/IP transport
R1# show ip route                ! expect: R 192.168.30.0/24 [120/1] via
172.16.0.2, Tunnel0
R1# ping 172.16.0.2              ! overlay link test
R1# traceroute 192.168.30.10     ! 172.16.0.2 then the host: the ISP never
appears as a hop
```

Part 4 — NAT and PAT

NAT rewrites private source addresses into public ones at the border (and reverses it for replies); PAT goes further and lets thousands of sessions share one public IP by also rewriting port numbers.

4.1 Vocabulary

Local means the address as seen from inside; global means as seen from outside.

Term	Meaning	Example in this lab
Inside local	Real private address of the inside host	192.168.10.10
Inside global	Public address that represents that host outside	203.0.113.20
Outside global	Real address of the outside host	198.51.100.10
Outside local	The outside host as seen from inside (usually the same)	198.51.100.10

4.2 Marking inside and outside interfaces

NAT only acts on packets moving between an `ip nat inside` interface and an `ip nat outside` interface. Forget either one and nothing is translated — the most common NAT fault.

```

! ===== Design A (ROAS): every subinterface that carries hosts is inside
=====
R1(config)# interface gi0/0.10
R1(config-subif)# ip nat inside
R1(config-subif)# interface gi0/0.20
R1(config-subif)# ip nat inside
R1(config-subif)# interface gi0/0.50
R1(config-subif)# ip nat inside
R1(config-subif)# exit
! Design B (MLS): the routed link to the MLS is inside instead:  interface
gi0/0 -> ip nat inside

! ===== ISP-facing interface is outside =====
R1(config)# interface gi0/1
R1(config-if)# ip nat outside
R1(config-if)# exit

```

Order of operations: for inside-to-outside traffic the router routes first, then translates; for outside-to-inside traffic it translates first, then routes. Replies to 203.0.113.x must therefore reach R1 — which is why the ISP has a static route for 203.0.113.0/24 pointing at R1 (Part 3.2).

4.3 Static NAT and port forwarding

Static NAT is a permanent, configured mapping that works in both directions, so outside hosts can start connections to an inside server. Use it for anything the outside must reach: web, DNS, mail.

```

! Option 1: one-to-one - the whole public IP belongs to one server, all ports
R1(config)# ip nat inside source static 192.168.50.2 203.0.113.6

! Option 2: port forwarding (static PAT) - one public IP, one service per
port
R1(config)# ip nat inside source static tcp 192.168.50.2 80 203.0.113.5 80
! web server, HTTP
R1(config)# ip nat inside source static udp 192.168.50.3 53 203.0.113.5 53
! DNS queries
R1(config)# ip nat inside source static tcp 192.168.50.3 53 203.0.113.5 53
! DNS over TCP (large replies); optional in PT

```

Read the syntax as: `ip nat inside source static <protocol> <inside local IP> <inside port> <inside global IP> <outside port>.`

- An outside user connecting to 203.0.113.5 on TCP 80 lands on the web server; UDP 53 on the same public IP lands on the DNS server.

- Every other port on 203.0.113.5 leads nowhere. Only what you forward is exposed — the security advantage over one-to-one mapping.
- HTTP runs over TCP, while ordinary DNS queries run over UDP. Match the protocol keyword to the service or the mapping never fires.
- This mapping is exactly why the DNS A-records in Part 2.6 point to 203.0.113.5.

4.4 Dynamic PAT (NAT overload) for whole departments

PAT maps many inside hosts onto one or a few public IPs by giving every session a unique source port, and the router tracks each session in its translation table. Use it for ordinary client traffic — every department PC browsing out.

Step 1 — an ACL that selects who gets translated. Here the ACL is a match list, not a firewall: `permit` means translate this. Anything not permitted (the implicit deny) is sent untranslated, and with a private source address it never receives a reply from the Internet.

```
R1(config)# access-list 1 remark KUET departments allowed to use PAT
R1(config)# access-list 1 permit 192.168.10.0 0.0.0.255      ! CSE
R1(config)# access-list 1 permit 192.168.20.0 0.0.0.255      ! EEE
! With the VLSM plan from 2.1 the wildcards follow the prefixes:
! access-list 1 permit 192.168.1.0 0.0.0.127      (CSE /25)
! access-list 1 permit 192.168.1.128 0.0.0.63      (EEE /26)
! access-list 1 permit 192.168.1.192 0.0.0.31      (ME /27)
```

Step 2a — PAT to a public pool.

```
R1(config)# ip nat pool KUET_POOL 203.0.113.20 203.0.113.30 netmask
255.255.255.0
R1(config)# ip nat inside source list 1 pool KUET_POOL overload    ! overload
= PAT
```

Step 2b — or PAT to the outside interface's own IP, the usual choice when the ISP gives you a single address.

```
R1(config)# ip nat inside source list 1 interface gi0/1 overload
```

What `overload` changes: without it, the pool is plain dynamic NAT — one public IP per inside host, first come first served, so an 11-address pool refuses the 12th host. With it, every host shares the pool and each session is told apart by its port, roughly 64,000 sessions per public IP.

The server VLAN is deliberately left out of ACL 1. The servers already have static mappings, and a static entry takes priority for its host anyway.

Verification

```

R1# show ip nat translations      ! the live table
R1# show ip nat statistics       ! inside/outside interfaces, hits, misses,
pool use
R1# clear ip nat translation *    ! flush dynamic entries (static ones
remain)
R1# debug ip nat                 ! every rewrite as it happens; stop with
undebug all

```

Sample table after the Outside PC browses the web server and a CSE PC pings out:

Pro	Inside global	Inside local	Outside local	Outside global
tcp	203.0.113.5:80	192.168.50.2:80	---	---
tcp	203.0.113.5:80	192.168.50.2:80	198.51.100.10:1025	198.51.100.10:1025
udp	203.0.113.5:53	192.168.50.3:53	---	---
icmp	203.0.113.20:1	192.168.10.10:1	198.51.100.10:1	198.51.100.10:1

Rows with --- in the outside columns are the permanent static mappings. Rows with outside addresses are live sessions: the second line is a web visit, the last is PAT at work — CSE's 192.168.10.10 appearing on the Internet as 203.0.113.20. Dynamic rows appear only after traffic flows and expire when idle.

Need	Use
Outside must start connections to a server	Static NAT (one-to-one) or port forwarding
Several public services behind one public IP	Port forwarding (static PAT)
Many clients going out, a few public IPs	Dynamic PAT with a pool + <code>overload</code>
Only one public IP from the ISP	PAT with <code>interface gi0/1 overload</code>

Part 5 — Device security and site-to-site VPN

Secure the device first (who may log in, from where, and how), then secure the traffic between sites; password recovery exists because physical console access overrides every password.

5.1 Console, VTY, enable and banner

Three doors need locks: the console port (physical access), the VTY lines (remote Telnet/SSH) and privileged EXEC (the enable password).

```
Router(config)# hostname R1
R1(config)# enable secret Kuet@Enable123          ! hashed; protects
privileged EXEC
R1(config)# service password-encryption          ! type 7: hides line
passwords in show run (weak, reversible)
R1(config)# security passwords min-length 8       ! optional
R1(config)# banner motd #Authorized access only. All activity on KUET network
devices is logged.#
R1(config)# line console 0
R1(config-line)# password Kuet@Con123
R1(config-line)# login                            ! actually require that
password
R1(config-line)# exec-timeout 5 0                 ! log out after 5 idle
minutes
R1(config-line)# logging synchronous
R1(config-line)# exit
R1(config)# line vty 0 4                          ! routers: 0 4 (some models
0 15); switches: 0 15
R1(config-line)# password Kuet@Vty123
R1(config-line)# login
R1(config-line)# exit
```

- `enable secret` is stored as a one-way hash; `enable password` is plain text (or weak type 7). If both exist, the secret wins — always use the secret.
- `password` without `login` does nothing: `login` is what makes the line ask for it.
- `service password-encryption` only defeats shoulder-surfing. Type 7 can be reversed with free tools.
- `banner motd` is shown before login. The delimiter (# here) can be any character that does not appear in the message. Word it as a legal warning, never as a welcome.
- Configure every VTY line that exists; one unprotected line is an open door.

5.2 SSH

Telnet sends everything, passwords included, in clear text; SSH encrypts the whole session. Each requirement has a reason:

1. **A non-default hostname and a domain name** — the RSA key is named `hostname.domain-name`, so IOS refuses to generate it while the hostname is still Router.

2. **An RSA key pair** — the device's cryptographic identity. SSH version 2 needs at least 768 bits; use 1024 or more.
3. **A local user account** — SSH logs in with a username and password, not a bare line password.
4. **VTY lines set to login local and transport input ssh** — check the local user database, and refuse Telnet.

```
R1(config)# ip domain-name kuet.ac.bd
R1(config)# username admin privilege 15 secret Kuet@Admin123    ! local user;
privilege 15 lands directly in enable mode
R1(config)# crypto key generate rsa
How many bits in the modulus [512]: 1024
R1(config)# ip ssh version 2
R1(config)# ip ssh time-out 60                                ! seconds allowed to
complete login
R1(config)# ip ssh authentication-retries 3
R1(config)# line vty 0 4
R1(config-line)# login local                                  ! use the username
database, ignore the line password
R1(config-line)# transport input ssh                          ! SSH only: Telnet is
refused
R1(config-line)# exec-timeout 5 0
```

A Layer-2 switch has no routed interfaces, so to reach it by SSH it also needs a management SVI and a default gateway:

```
SW1(config)# interface vlan 50                                ! a management VLAN, never
VLAN 1
SW1(config-if)# ip address 192.168.50.20 255.255.255.0
SW1(config-if)# no shutdown
SW1(config-if)# exit
SW1(config)# ip default-gateway 192.168.50.1                ! lets the switch answer
admins in other subnets
```

Test from a Packet Tracer PC: Desktop → Command Prompt → `ssh -l admin 192.168.10.1`. Verify on the router with `show ip ssh` (enabled, version 2) and `show ssh` (active sessions).

5.3 Password recovery on an ISR router (ROMMON)

The 16-bit configuration register tells the router how to boot. The normal value 0x2102 means: load IOS from flash, then load startup-config from NVRAM. 0x2142 adds bit 6 (0x0040), which means ignore NVRAM. The router boots with an empty configuration —

no passwords — while the old configuration sits untouched in NVRAM. Nothing is erased, which is why the procedure is safe.

Register	Boot behaviour
0x2102	Normal: IOS from flash, load startup-config, Break ignored once running
0x2142	IOS from flash, skip startup-config (bypass NVRAM)

1. Connect a PC to the console port (PT: console cable, then PC Desktop → Terminal).
2. Power-cycle the router (PT: Physical tab → power switch off, then on).
3. Within about 60 seconds of power-on, send a Break (Ctrl+Break in the PT terminal).
The prompt becomes `rommon 1 >`.
4. Change the register and reboot, then work through the configuration below.

```
rommon 1 > confreg 0x2142          ! ignore NVRAM at next boot
rommon 2 > reset                   ! reboot now

--- System Configuration Dialog ---
Would you like to enter the initial configuration dialog? [yes/no]: no

Router> enable                     ! no password: the config is
empty
Router# copy startup-config running-config ! NVRAM -> RAM. NEVER the
reverse here: it would overwrite the saved config
Destination filename [running-config]? <Enter>
R1#                                ! the old hostname returns:
the old config is loaded
R1# configure terminal
R1(config)# no enable secret       ! remove the forgotten secret
R1(config)# enable secret Kuet@NewPass123 ! and set a new one
R1(config)# line console 0
R1(config-line)# password Kuet@NewCon123 ! reset line passwords too if
they are unknown
R1(config-line)# exit
R1(config)# interface gi0/0
R1(config-if)# no shutdown         ! interfaces come back
administratively down
R1(config-if)# interface gi0/1
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# config-register 0x2102 ! restore normal boot
```

```

R1(config)# end
R1# copy running-config startup-config           ! save the new passwords
R1# show version                                ! last line: register 0x2142
(will be 0x2102 at next reload)
R1# reload                                       ! optional: confirm a normal
boot with the new password

```

- **Why the interfaces stay down.** In the blank boot every interface sits in its default, shut state. The copy merges commands, and no shutdown is never saved as a line, so nothing turns the interfaces back on.
- **Why the direction matters.** `copy startup-config running-config` pulls the old config into memory. The reverse copy would save the empty configuration over the real one.
- **The classic mistake** is forgetting `config-register 0x2102`: the next reboot ignores NVRAM again and the router looks as if it lost its whole configuration.
- Catalyst 2960 switches use a different method (hold the Mode button while powering on, then `flash_init` and rename `config.text`), so `confreg` does not apply to them.

5.4 GRE over IPsec site-to-site VPN

Why combine them. GRE carries anything, including RIP's multicast updates, but encrypts nothing. Crypto-map IPsec encrypts, but only carries unicast IP, so routing protocols cannot run over it. GRE over IPsec gets both: GRE builds the tunnel and carries the routing, and IPsec encrypts the GRE packets as they leave the router. Build it on top of the Part 3.5 tunnel — keep the underlay statics, Tunnel0 and RIPv2 exactly as they are.

Prerequisite in Packet Tracer: ISR routers ship without the security feature set. If crypto commands are rejected, enable the security license first: `license boot module c2900 technology-package securityk9` (c1900 on a 1941), accept, save and reload.

Packet walk on R1:

1. PC 192.168.10.10 sends to 192.168.30.10. R1's RIP route says: via 172.16.0.2, out Tunnel0.
2. Tunnel0 encapsulates it: new outer header 200.1.1.1 → 200.1.2.1, protocol GRE (47).
3. R1 routes that GRE packet: 200.1.2.1 matches the default route, so it leaves through G0/1.
4. On G0/1 the crypto map checks its ACL. GRE from 200.1.1.1 to 200.1.2.1 matches, so the packet is encrypted with ESP (protocol 50) and sent to the ISP.
5. R2 decrypts it, removes the GRE header and routes the original packet into its LAN.

Phase 1 — ISAKMP (IKE) policy

The peers authenticate each other and build a secure management channel, the ISAKMP SA, over UDP 500. Every parameter must match on both peers — remember HAGLE: Hash, Authentication, Group, Lifetime, Encryption.

```
R1(config)# crypto isakmp policy 10          ! priority: lower numbers
are offered first
R1(config-isakmp)# encryption 3des          ! E: cipher protecting the
Phase 1 channel
R1(config-isakmp)# hash md5                 ! H: integrity check on
Phase 1 messages
R1(config-isakmp)# authentication pre-share ! A: both peers hold the
same secret key
R1(config-isakmp)# group 5                  ! G: Diffie-Hellman group 5
(1536-bit) key exchange
R1(config-isakmp)# lifetime 86400           ! L: seconds before re-
keying (the default)
R1(config-isakmp)# exit
R1(config)# crypto isakmp key Kuet@VPNkey address 200.1.2.1 ! shared key,
bound to the PEER's WAN IP
```

- **3DES** is the symmetric cipher (168-bit key). **MD5** is the hash that proves messages were not altered. Both are legacy and appear because Packet Tracer supports them; production designs use AES-256, SHA-256 and DH group 14 or higher, often with IKEv2.
- **DH group 5** lets the two peers derive the same secret key across the untrusted Internet without ever sending it.
- **Pre-share** authenticates with that shared key. The `address` must be the peer's IP exactly as the peer sources its packets — its physical WAN IP.

Phase 2 — transform set

Phase 2 negotiates how the user data itself is protected. The transform set must match on both peers.

```
R1(config)# crypto ipsec transform-set VPN-SET esp-3des esp-md5-hmac
!                               name      encryption integrity (HMAC-
MD5)
R1(cfg-crypto-trans)# exit
! Real IOS option inside the transform set: mode transport (saves 20 bytes,
because GRE already adds an outer header)
```

ESP (IP protocol 50) provides both encryption and integrity for the data.

The crypto ACL matches GRE between the WAN IPs, not LAN to LAN

```
R1(config)# access-list 110 permit gre host 200.1.1.1 host 200.1.2.1
R2(config)# access-list 110 permit gre host 200.1.2.1 host 200.1.1.1    !
exact mirror on R2
```

The crypto map inspects packets at the physical interface, after routing and after GRE encapsulation (step 4 of the packet walk). By then 192.168.10.10 and 192.168.30.10 are buried inside the GRE payload. The only header the crypto map can see is the outer one: protocol GRE, 200.1.1.1 to 200.1.2.1.

An ACL written as `permit ip 192.168.10.0 0.0.0.255 192.168.30.0 0.0.0.255` would therefore never match. The GRE packets would leave unencrypted and the IPsec SA would never form. Matching GRE also means one entry encrypts everything in the tunnel — every LAN and the RIP updates themselves — and adding a new LAN later needs no crypto change.

The crypto map goes on the physical interface, not the tunnel

```
R1(config)# crypto map VPN-MAP 10 ipsec-isakmp
R1(config-crypto-map)# description Encrypt GRE to R2
R1(config-crypto-map)# set peer 200.1.2.1                ! R2's WAN IP
R1(config-crypto-map)# set transform-set VPN-SET
R1(config-crypto-map)# match address 110                 ! the GRE crypto ACL
R1(config-crypto-map)# exit
R1(config)# interface gi0/1
R1(config-if)# crypto map VPN-MAP                        ! physical egress
interface, NOT Tunnel0
```

1. The physical interface is where the finished GRE packet actually leaves the router — the last point where it can be encrypted.
2. On Tunnel0 the router only sees the original LAN packets before encapsulation, so an ACL matching GRE could never match there.
3. IKE runs between the WAN addresses: `set peer` and the key's `address` are both WAN IPs, and the peer expects IKE packets from the physical interface it knows.

Modern IOS offers an alternative — an IPsec profile attached to the tunnel with `tunnel protection ipsec profile` — but Packet Tracer labs use crypto maps.

R2 — the mirror configuration

```

R2(config)# crypto isakmp policy 10
R2(config-isakmp)# encryption 3des
R2(config-isakmp)# hash md5
R2(config-isakmp)# authentication pre-share
R2(config-isakmp)# group 5
R2(config-isakmp)# lifetime 86400
R2(config-isakmp)# exit
R2(config)# crypto isakmp key Kuet@VPNkey address 200.1.1.1    ! points back
at R1's WAN IP
R2(config)# crypto ipsec transform-set VPN-SET esp-3des esp-md5-hmac
R2(cfg-crypto-trans)# exit
R2(config)# access-list 110 permit gre host 200.1.2.1 host 200.1.1.1
R2(config)# crypto map VPN-MAP 10 ipsec-isakmp
R2(config-crypto-map)# set peer 200.1.1.1
R2(config-crypto-map)# set transform-set VPN-SET
R2(config-crypto-map)# match address 110
R2(config-crypto-map)# exit
R2(config)# interface gi0/1
R2(config-if)# crypto map VPN-MAP

```

The SAs are built on demand: nothing happens until the first matching packet. RIP's 30-second updates are GRE packets too, so the VPN usually builds itself within a minute; a user's first ping may lose a packet or two while IKE negotiates.

```

R1# show crypto isakmp sa      ! QM_IDLE = Phase 1 complete (good);
MM_NO_STATE = Phase 1 failing
R1# show crypto ipsec sa      ! encaps/encrypt and decaps/decrypt counters
should climb on both peers
R1# show crypto map          ! peer, ACL, transform set and the interface it
is bound to

```

If the VPN will not form, check in this order: WAN-to-WAN ping over the underlay; ISAKMP policies match (HAGLE); the key's address is the peer's WAN IP; transform sets match; the ACLs are mirror images; the crypto map sits on the physical outside interface of both routers.

Troubleshooting toolkit

Work bottom-up: interface state first, then Layer 2 (VLANs, trunks), then addressing, then routing, then services (DHCP, NAT, VPN) — most lab faults are a shut interface or a missing VLAN, not a routing bug.

Interface state

```
R1# show ip interface brief
```

Interface	IP-Address	OK?	Method	Status
Protocol				
GigabitEthernet0/0	unassigned	YES	unset	up
GigabitEthernet0/0.10	192.168.10.1	YES	manual	up
GigabitEthernet0/1	200.1.1.1	YES	manual	up
Tunnel0	172.16.0.1	YES	manual	up

Status / Protocol	Meaning	Fix
up / up	Working	Nothing to fix
administratively down / down	shutdown is configured	no shutdown
down / down	Layer 1: no cable, wrong cable, far end off	Check the cable and the neighbour
up / down	Layer 2: encapsulation, clock rate or keepalive mismatch	Compare both ends of the link

Reading the routing table

```
R1# show ip route
```

Codes: L - local, C - connected, S - static, R - RIP, O - OSPF,
 IA - OSPF inter area, E1 - OSPF external type 1, E2 - OSPF external type 2,
 * - candidate default
 Gateway of last resort is 200.1.1.2 to network 0.0.0.0

```
S* 0.0.0.0/0 [1/0] via 200.1.1.2
C 192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L 192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
R 192.168.30.0/24 [120/1] via 172.16.0.2, 00:00:14, Tunnel0
```

```
MLS1# show ip route (Design B)
```

```
O*E2 0.0.0.0/0 [110/1] via 10.0.0.1, 00:12:03, GigabitEthernet0/1
```

Anatomy of a learned route, using the R line: code R (learned by RIP) → destination 192.168.30.0/24 → [120/1] = administrative distance 120 / metric 1 hop → next hop 172.16.0.2 → 00:00:14 since the last update → exit interface Tunnel0. For OSPF the metric is cost; for static routes it is 0.

Code	Meaning	Where it appears in these labs
C	Directly connected network	Every configured interface that is up
L	Local /32 — the router's own IP on that interface	Paired with every C route
S	Static route	Underlay routes; the ISP's route to 203.0.113.0/24
S*	Static default route (candidate default)	R1 and R2 toward the ISP
O	OSPF intra-area route	MLS VLAN subnets as learned by R1
O IA	OSPF inter-area route	Multi-area designs only
O E2	OSPF external type 2: redistributed in, metric fixed at the seed	Routes redistributed into OSPF
O*E2	Default route injected by default-information originate	Every OSPF router except R1
R	RIP route	Branch LAN via Tunnel0 on R1, HQ LANs via Tunnel0 on R2

Gateway of last resort is not set means there is no default route: anything headed for the Internet is dropped.

Protocol and service checks

Command	Healthy result	Bad sign and likely cause
<code>show ip ospf neighbor</code>	State FULL (FULL/DR or FULL/BDR on Ethernet; 2WAY/DROTHER between two DROTHERs is normal)	Empty: area, subnet, timers or passive interface mismatch. Stuck in EXSTART/EXCHANGE: MTU mismatch. INIT: hellos only one way
<code>show ip protocols</code>	Right networks, passive interfaces, RIP version 2, auto-summary off	A missing network statement or the wrong version
<code>show ip nat translations</code>	Static rows always; dynamic PAT rows after traffic	Empty after pinging out: missing <code>ip nat inside</code> or <code>outside</code> , or the ACL does not match the source

Command	Healthy result	Bad sign and likely cause
<code>show ip nat statistics</code>	Hits climbing; correct inside and outside interfaces listed	Misses climbing; interfaces missing
<code>show crypto isakmp sa</code>	QM_IDLE, status ACTIVE	MM_NO_STATE: Phase 1 mismatch, wrong key or wrong peer address
<code>show crypto ipsec sa</code>	encaps/encrypt and decaps/decrypt climbing on both peers	encrypt climbing but decrypt at 0: the far end is not sending back (ACLs not mirrored)
<code>show interfaces tunnel 0</code>	up/up with correct source and destination	up/down: no route to the destination. RECURDOWN log: the WAN is being advertised through the tunnel
<code>show interfaces trunk</code>	Needed VLANs in the forwarding and not pruned list	VLAN missing: not created, not allowed, or blocked by STP
<code>show vlan brief</code>	Every access port in its intended VLAN	Ports still in VLAN 1
<code>show ip dhcp binding</code>	A lease for each client MAC	None: pool network mismatch, missing relay, or wrong default-router

A healthy `show crypto ipsec sa` excerpt from R1:

```
interface: GigabitEthernet0/1
  Crypto map tag: VPN-MAP, local addr 200.1.1.1
  local  ident (addr/mask/prot/port): (200.1.1.1/255.255.255.255/47/0)
  remote ident (addr/mask/prot/port): (200.1.2.1/255.255.255.255/47/0)
  current_peer 200.1.2.1 port 500
    #pkts encaps: 42, #pkts encrypt: 42, #pkts digest: 0
    #pkts decaps: 40, #pkts decrypt: 40, #pkts verify: 0
```

The 47 in both ident lines is the GRE protocol number — proof that the SA protects exactly the traffic the crypto ACL named. Encaps and decaps both rising means traffic is being encrypted in both directions.

Debugs (use sparingly, and always finish with `undebg all`): `debug ip ospf adj`, `debug ip rip`, `debug ip nat`, `debug crypto isakmp`.

Host side (PT Desktop → Command Prompt): `ipconfig /all`, `ipconfig /release` then `ipconfig /renew`, `ping`, `tracert`, `nslookup www.kuet.ac.bd`.

Packet Tracer: the Fast Forward Time button

Packet Tracer simulates real protocol timers. After you cable a switch, each port spends 15 s in STP listening and 15 s in learning before it forwards, so its link light stays amber for about 30 s. A PC that asks for DHCP inside that window gets no answer and falls back to a 169.254.x.x address. OSPF adjacencies, RIP updates and ARP also take simulated seconds.

The **Fast Forward Time** button sits in the time bar at the bottom of the workspace, beside the clock and Power Cycle Devices. It jumps the simulation clock forward so pending timers finish at once. Use it:

- after cabling, until the switch port lights turn green;
- before clicking DHCP on the PCs — or click DHCP again after fast-forwarding;
- after configuring OSPF or RIP, before checking `show ip route` ;
- when the first pings time out while ARP resolves, or while the VPN SAs negotiate.

If a PC still shows 169.254.x.x after fast-forwarding, the fault is real: check the pool, the helper address and the port's VLAN, not the timer. PortFast on host ports removes the STP delay permanently.

Where to use what — decision cheat sheet

Pick the tool by the problem it solves: every technology in these labs exists to fix one specific limitation of the layer below it.

If you need to...	Use	Why
Separate groups of hosts on one switch	VLANs + access ports	Separate broadcast domains
Carry many VLANs between switches or to a router	802.1Q trunk, native VLAN 99, DTP off	One link, tagged frames, no hopping
Route between a few VLANs cheaply	Router-on-a-stick	One router port, one subinterface per VLAN
Route between many busy VLANs	Multilayer switch SVIs + <code>ip routing</code>	Hardware-speed routing
Split one block into unequal subnets	VLSM, largest first	No waste, no overlap
Give addresses to one small subnet	IOS DHCP pool on its router	No extra server needed

If you need to...	Use	Why
Serve DHCP to other subnets or sites	<code>ip helper-address</code> on each client gateway	Broadcasts do not cross routers
Let outsiders reach an inside server by name	Static NAT or port forwarding + DNS A-record pointing to the public IP	Fixed, two-way mapping; private IPs are unroutable outside
Let many inside clients reach the Internet	Dynamic PAT (<code>overload</code>) with an ACL	Many hosts, few public IPs
Dynamic routing inside a larger network	OSPF	Fast, classless, scalable
Simple routing, or routing inside a tunnel	RIPv2 with <code>no auto-summary</code>	Minimal configuration
One exit to the ISP	Static default + <code>default-information originate</code>	Nothing else to learn; OSPF spreads it
Join private LANs across the Internet	GRE overlay on a static underlay	The ISP never sees private routes
Encrypt that connection	IPsec crypto map on the physical interface, ACL matching GRE	Encrypts after encapsulation
Manage devices remotely	SSH v2, <code>login local</code> , <code>transport input ssh</code>	Encrypted, per-user login
Recover a lost enable secret	ROMMON, <code>confreg 0x2142</code>	Bypasses NVRAM without erasing it

Exam and viva traps

1. On a 3560, `switchport trunk encapsulation dot1q` must come before `switchport mode trunk` ; on a 2960 the command does not exist.
2. On a subinterface, `encapsulation dot1q` must come before `ip address` .
3. `ip routing` is off by default on a multilayer switch.
4. Wildcard = 255.255.255.255 – subnet mask. OSPF process IDs are local; areas must match.
5. `default-information originate` does nothing unless the router has a default route (or uses `always`).
6. RIP `network` statements are classful; use `no auto-summary` with VLSM or discontinuous networks.

7. Never advertise the tunnel destination through the tunnel — that is recursive routing.
8. `ip helper-address` goes on the client-facing interface, not the server side.
9. NAT needs both `ip nat inside` and `ip nat outside`, and the ISP must route the public block back.
10. `overload` is what turns dynamic NAT into PAT.
11. During recovery, `copy startup-config running-config` — never the reverse — then `config-register 0x2102`.
12. The crypto ACL matches GRE between WAN IPs, and the crypto map goes on the physical interface.